

PROJECT REPORT

STUDENT EXPENSE TRACKER

A beginner-friendly Python application for recording and analysing student expenses

Submitted by	Sakshi leelani
Registration Number	26BAI11107
Course / Subject	Introduction to Problem Solving
Faculty	DR PRIYADARSHINI B
Academic Year	2026–2030

Project Report

Table of Contents

1. Introduction
2. Problem Statement
3. Objectives and Scope
4. Functional Requirements
5. Non-Functional Requirements
6. System Architecture
7. Design Diagrams
8. Design Decisions and Rationale
9. Implementation Details
10. Screenshots / Sample Results
11. Testing Approach
12. Challenges Faced
13. Learnings and Key Takeaways
14. Future Enhancements
15. Conclusion
16. References

Report accuracy note: This report describes the Python script supplied for the project. Sample outputs are illustrative, not screenshots of an executed test. The current code uses a single Python script and a CSV file; the modular structure and further validation listed under future enhancements are not claimed as already implemented.

1. Introduction

Students often make small daily purchases such as food, travel, stationery, mobile recharges and personal items. When these purchases are not recorded, it can be difficult to understand where money is going or whether spending is within a monthly budget.

The Student Expense Tracker is a menu-driven command-line application developed in Python. It allows a student to enter expense details, view saved records, delete a selected record, see spending grouped by category, compare total recorded spending with a budget, and review a simple spending-level message. Expense records are stored in a CSV file so that they remain available after the program closes.

The project applies basic Python concepts including functions, loops, conditional statements, dictionaries, file handling, CSV processing, numeric conversion and formatted output.

2. Problem Statement

Students may not maintain a consistent record of their expenses. Without a simple record, they may find it difficult to calculate total spending, compare expenses across categories, or check their spending against a planned budget. The project addresses this problem by providing a small local program to record and summarise expenses using a CSV file.

3. Objectives and Scope

Objectives

- Provide a simple menu for common expense-tracking tasks.
- Record the date, category, amount and description of each expense.
- Allow saved expense records to be viewed and deleted by record number.
- Calculate totals for each category.
- Compare total recorded expenses with a user-entered budget.
- Provide a basic spending summary to help the user review their records.

Scope

The current version is a local, single-user, text-based Python program. It stores data in **expenses.csv** in the program's working directory. It does not require an online account or database server. It currently calculates total spending from all rows in the file; it does not filter spending by month, even though the budget prompt says "monthly budget." This limitation should be considered when interpreting the budget report.

Target Users

The primary target users are students who want a basic way to record personal expenses on a computer with Python installed.

4. Functional Requirements

The following functional requirements are based on the supplied code.

ID	Requirement	Implementation in supplied code
FR-01	Add expense	Prompts for date, category, amount and description, then appends a row to expenses.csv.
FR-02	View expenses	Reads the CSV file and prints each non-empty record.
FR-03	Delete expense	Loads records, displays them, accepts a 1-based record number and rewrites the CSV without that row.

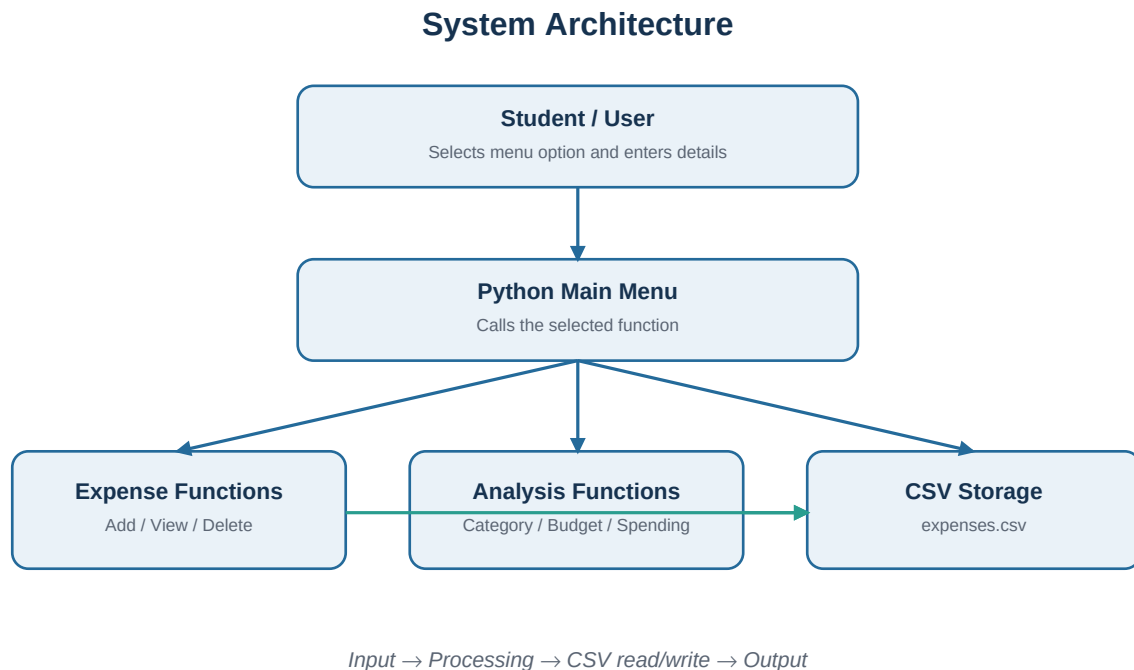
ID	Requirement	Implementation in supplied code
FR-04	Category summary	Uses a dictionary to add amounts for each category.
FR-05	Budget analysis	Asks for a budget, calculates total recorded expenses and displays the remaining amount or excess.
FR-06	Spending analysis	Displays total spending and a simple Low / Moderate / High message using fixed thresholds.
FR-07	Menu navigation	Repeats the main menu until the user selects Exit.
FR-08	Persistent storage	Uses CSV file reading and writing to retain records between runs.

5. Non-Functional Requirements

ID	Requirement	Expected behaviour / current status
NFR-01	Usability	The numbered menu and short prompts should make common actions easy to find. This is a text-based interface.
NFR-02	Reliability	CSV records are retained between program runs. The current implementation handles a missing file in view, delete, category summary and total calculation.
NFR-03	Error handling	Invalid menu choices and out-of-range delete numbers are handled. Non-numeric amounts, malformed rows and invalid numeric input can still cause errors; stronger validation is needed.
NFR-04	Maintainability	Functions separate major operations, making the single script easier to read. Moving functions into separate files would improve modularity further.
NFR-05	Performance	For a student-sized CSV file, the program reads records sequentially. Runtime and memory use increase with the number of records.
NFR-06	Resource efficiency	The program uses the Python standard library and a local CSV file, with no database server or external service.

6. System Architecture

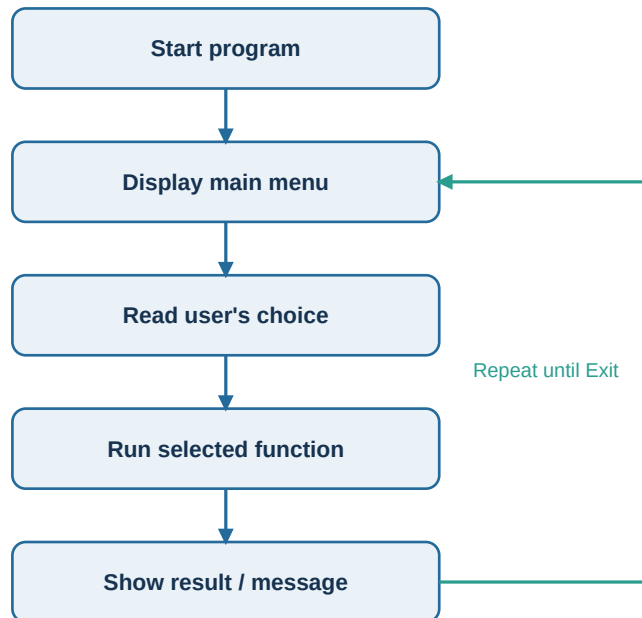
The current application follows a simple flow: the user selects an option from the menu, the main function calls the corresponding operation, and the operation reads from or writes to the CSV file as needed. The result is printed in the terminal. This is a logical architecture of the supplied script, not a claim that separate modules already exist.



7. Design Diagrams

7.1 Process Workflow

Process Workflow

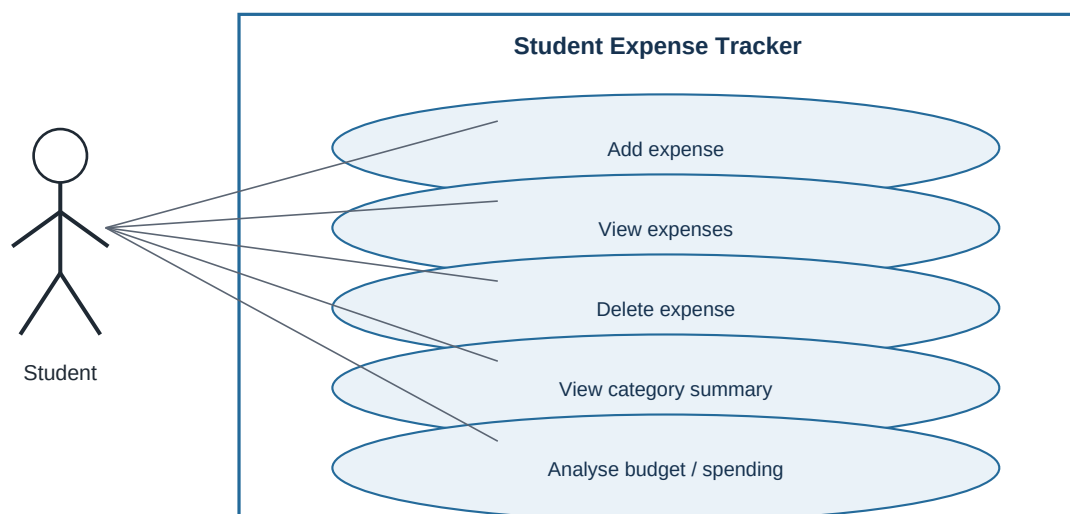


Exit option ends the loop and closes the program.

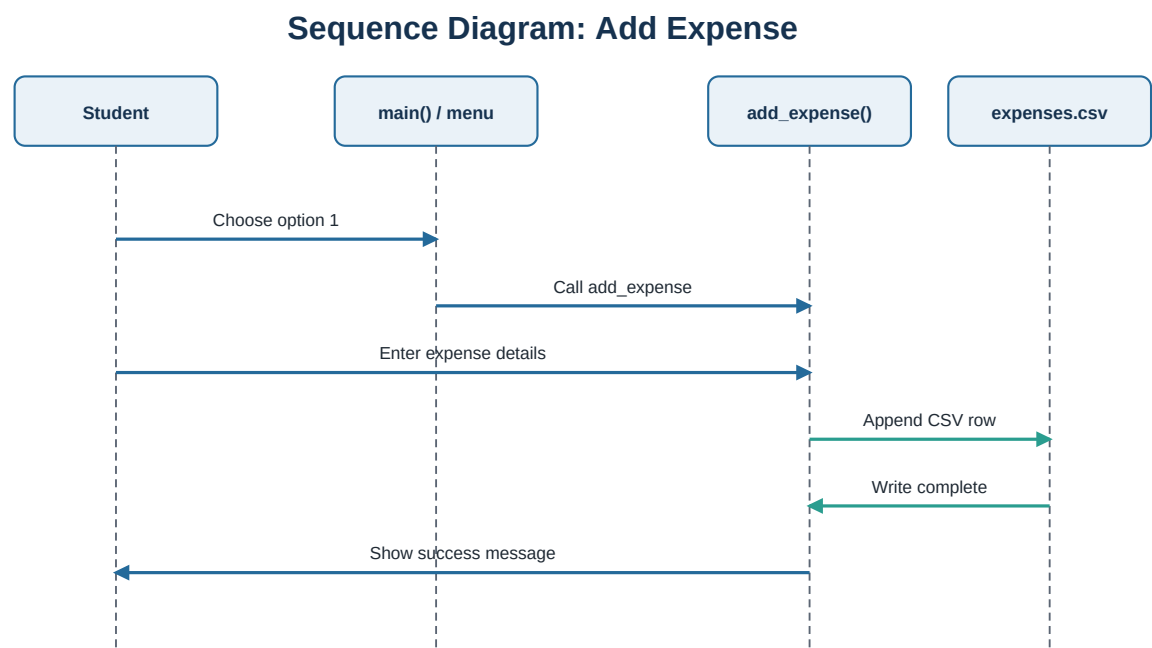
The menu repeats after each operation. Selecting option 7 prints a closing message and ends the loop.

7.2 Use Case Diagram

Use Case Diagram (simplified)

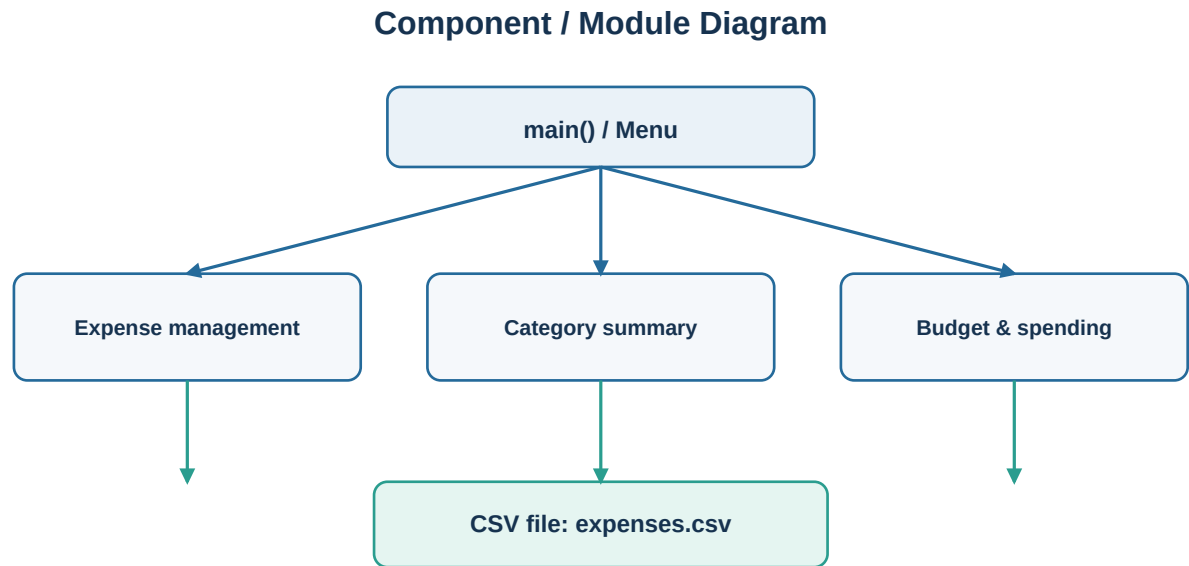


7.3 Sequence Diagram — Add Expense



The sequence shows the user selecting Add Expense, entering the four fields, and the function appending the record to the CSV file before displaying a success message.

7.4 Component Diagram



7.5 Storage Schema / ER Diagram

Storage Schema / ER Diagram

EXPENSE (CSV record)
date : DD-MM-YYYY
category : text
amount : decimal number
description : text

One CSV row represents one expense; no separate relational tables are used.

The current storage design is a flat CSV file rather than a relational database. Each row stores one expense with four fields in this order: date, category, amount and description. There is no separate user table, category table or database primary key. During deletion, the program identifies a record by its displayed row number.

8. Design Decisions and Rationale

Python: Chosen because its syntax is beginner-friendly and it supports functions, dictionaries, loops, conditions and file handling.

CSV storage: The built-in csv module provides a simple format that can be opened with common spreadsheet software and does not require database setup.

Functions: Each major operation is placed in a function so the menu can call it independently.

Dictionary for categories: A dictionary maps each category name to the sum of its expense amounts.

Command-line interface: Keeps the first version small and focuses on program logic rather than graphical design.

9. Implementation Details

9.1 Technologies and Libraries

- Python 3
- **csv** — reads and writes CSV records
- **datetime** — imported in the supplied code, but not currently used
- **expenses.csv** — local data file created when an expense is added

9.2 Main Functions

Function	Purpose	Key concepts used
<code>add_expense()</code>	Collects four fields and appends one record.	<code>input()</code> , <code>open(..., 'a')</code> , <code>csv.writer</code> , <code>writerow()</code>
<code>view_expenses()</code>	Prints saved records or a no-records message.	<code>try/except</code> , <code>csv.reader</code> , <code>loop</code> , <code>f-strings</code>
<code>delete_expense()</code>	Removes the selected row and rewrites the file.	<code>list()</code> , <code>pop()</code> , <code>open(..., 'w')</code> , <code>writerows()</code>
<code>category_summary()</code>	Totals amounts grouped by category.	<code>dictionary</code> , <code>if/else</code> , <code>float conversion</code>
<code>total_expense()</code>	Returns the sum of all recorded amounts.	<code>accumulator</code> , <code>file reading</code> , <code>return</code>
<code>budget_analysis()</code>	Compares a user-entered budget to total recorded spending.	<code>arithmetic</code> , <code>conditional statements</code>
<code>spending_analysis()</code>	Prints total spending and a threshold-based label.	<code>if/elif/else</code>
<code>main()</code>	Displays menu and dispatches choices until Exit.	<code>while loop</code> , <code>if/elif/else</code>

9.3 Data Format

```
DD-MM-YYYY,Category,Amount,Description
29-09-2026,Food,120.50,Evening snacks
29-09-2026,Travel,40,Bus fare
```

The lines above are illustrative examples of the CSV format, not actual records from the user's computer.

9.4 Important implementation notes

- The date is accepted as text; the imported `datetime` module is not used to validate the date.
- The amount is converted to float, but the current code does not reject negative amounts or handle text that cannot be converted to a number.
- The budget calculation uses every record in `expenses.csv`, not only records from the current month.
- The spending labels use fixed thresholds: below **■**3,000 is Low; **■**3,000 to below **■**7,000 is Moderate; **■**7,000 or more is High. These are program-defined labels, not financial advice.
- The delete function uses a row number, not a permanent unique expense ID.

10. Screenshots / Sample Results

The examples below illustrate the expected style of terminal output. Replace or supplement them with screenshots captured from your own run before submitting the report.

10.1 Main Menu

```
=====
          STUDENT EXPENSE TRACKER
=====
1. Add Expense
2. View Expenses
3. Delete Expense
4. Category Summary
5. Budget Analysis
6. Spending Analysis
7. Exit
=====
Enter your choice:
```

10.2 Add Expense — illustrative interaction

```
Enter date (DD-MM-YYYY): 29-09-2026
Enter category: Food
Enter amount: █120.50
Enter description: Evening snacks
Expense added successfully!
```

10.3 Category Summary — illustrative output

```
----- CATEGORY SUMMARY -----
Food: █120.50
Travel: █40.00
```

10.4 Budget Analysis — illustrative output

```
----- BUDGET REPORT -----
Monthly Budget : █5000.00
Total Spent    : █160.50
Remaining      : █4839.50
You are within your budget.
```

These sample values are hypothetical and assume only the two example expenses are stored. They do not demonstrate a verified run of the code.

11. Testing Approach

Testing should be performed by running the program in a terminal and checking that actual output matches the expected behaviour. The table is a test plan; tests should be marked Pass only after they have been executed.

Test ID	Test scenario	Expected result	Status
T-01	Add a valid expense	Success message appears and a row is appended to expenses.csv.	Not run
T-02	View when file has records	Each record is printed with date, category, amount and description.	Not run
T-03	View when file is missing or empty	Program prints a no-expenses message.	Not run
T-04	Delete using a valid row number	Selected row is removed from the CSV file.	Not run
T-05	Delete using 0 or a number above record count	Program prints an invalid-number message and does not delete a row.	Not run

Test ID	Test scenario	Expected result	Status
T-06	Category summary with repeated categories	Amounts in the same category are added together.	Not run
T-07	Budget above total expenses	Remaining amount is positive and within-budget message appears.	Not run
T-08	Budget below total expenses	Budget-exceeded message shows the excess.	Not run
T-09	Enter letters instead of an amount	Current code may raise ValueError; validation should be added.	Known limitation
T-10	Select an invalid menu option	Program prints an invalid-choice message and shows menu again.	Not run

12. Challenges Faced

The following are likely implementation challenges based on the supplied code and can be edited to reflect the challenges actually experienced during development:

- Keeping the CSV column order consistent when saving and reading records.
- Converting amount text to a numeric value so totals can be calculated.
- Updating the CSV safely when a record is deleted.
- Handling the case where the CSV file does not yet exist.
- Making sure the budget and spending summaries are interpreted correctly, especially because the current calculation includes all saved records.

13. Learnings and Key Takeaways

This project provides practice in decomposing a small real-world task into functions, collecting user input, using conditional logic and loops, reading and writing files, processing CSV data, and using a dictionary to aggregate information. It also demonstrates why validation, test cases, documentation and clear data definitions matter in a practical program.

14. Future Enhancements

The following improvements are recommended for later versions and are not all present in the current code:

1. Validate the date using datetime and require the DD-MM-YYYY format.
2. Validate amounts and reject negative values or non-numeric input.
3. Filter expenses by month so the monthly budget is compared with spending for the selected month.
4. Add an edit-expense feature.
5. Add a unique ID to each record so deletion does not depend on row position.
6. Split the script into separate modules such as `main.py`, `expense_manager.py`, `category_manager.py`, `budget.py` and `storage.py`.
7. Add automated tests for calculation and file-handling functions.
8. Add optional CSV export/reporting or a simple graphical user interface.
9. Consider backup handling so users can recover from accidental changes to the CSV file.

15. Conclusion

The Student Expense Tracker is a small Python application designed to help students record expenses and review their spending. The current version supports adding, viewing and deleting expenses, category-wise totals, budget comparison and a basic spending analysis. CSV storage keeps the implementation straightforward for beginners. The project can be strengthened by adding input validation, month-based calculations, automated tests and a modular file structure before final submission.

16. References

1. VITyarthi, *Build Your Own Project — General Project Instructions & Submission Guidelines*, user-provided project PDF, 7 pages.
2. Python Software Foundation, *Python 3 Documentation* — <https://docs.python.org/3/>
3. Python Software Foundation, *csv — CSV File Reading and Writing* — <https://docs.python.org/3/library/csv.html>
4. Supplied Student Expense Tracker source code provided by the project author.

Before submitting: run the test plan and update the statuses, add real screenshots, and make sure the GitHub repository includes `README.md`, `statement.md`, organized source code and the report PDF as required by the course guidelines.